

How every ID and name is born.

The single source of truth for identifier generation in Saadaa CreatorHub — SIF, Post (P), Collab (C), nomenclature, and campaign IDs. Every rule in plain English *and* exact logic, for both the live project and the historic archive.

v2 • 2026-06-25

8 sections • 24 rules

New + Historic

Verified vs live DB

Anatomy of an ID

Every identifier is built on the creator's **SIF**. A deliverable adds a **-P** number; a collaboration adds a **-C** number. The two live in *separate columns* — a `post_id` never contains a **-C**, and a `collab_id` never contains a **-P**.

Read the parts

Worked on a real creator, SIF-1007, with two historic collabs (C1, C2 / P1, P2), onboarded again with 3 deliverables.

SIF-1007

CREATOR

- P3

DELIVERABLE

post_id — one per reel / static post. The **P** climbs per creator across every collab. Here P3 because P1-P2 are already used in history. **No -C lives here.**

SIF-1007

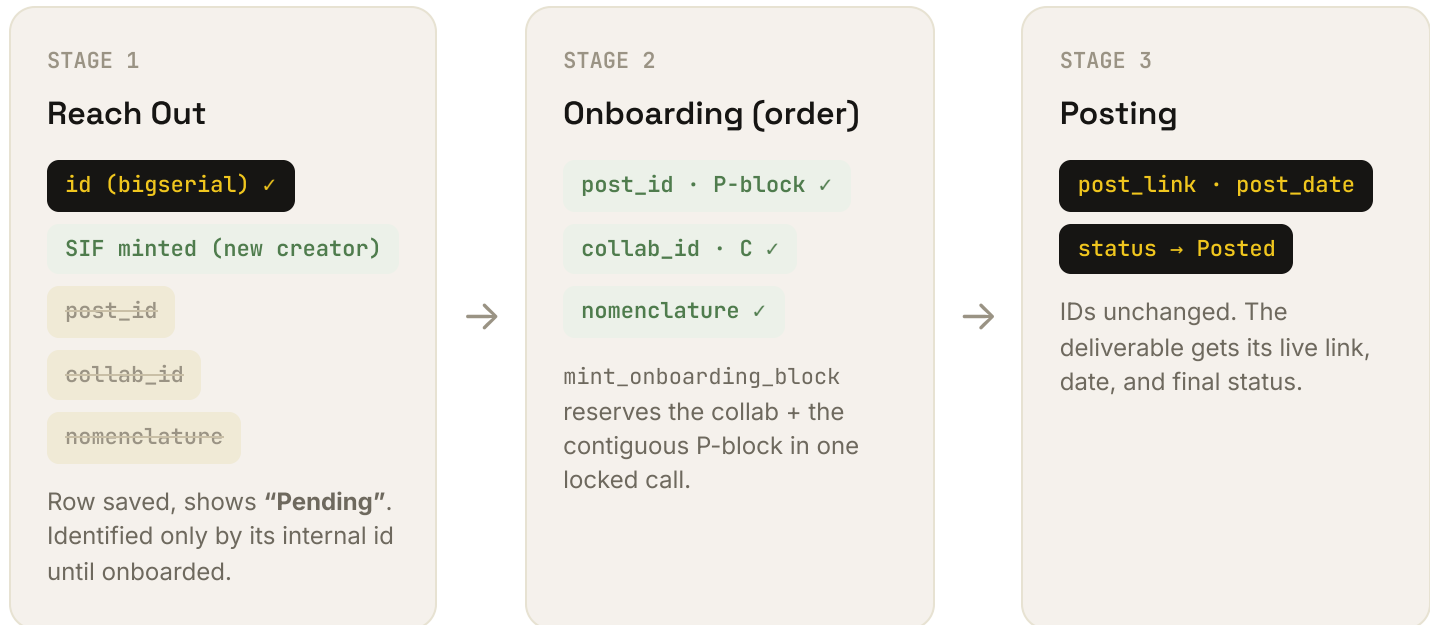
CREATOR

- C3COLLAB /
ORDER

collab_id — one per order. All deliverables of that order share it. C3 because the creator already had C1, C2 historically. The 3 new deliverables P3/P4/P5 all carry `collab_id` **SIF-1007-C3**.

When each ID is minted

The cardinal rule of the new project: **reach-out creates the row but mints nothing**. The Post (P) and Collab (C) IDs are both born at **onboarding**, the moment an order is attached.



1

The SIF Creator ID

The SIF id is the root of the entire naming system — every `post_id` and `collab_id` is built on top of it. It is the only global linear counter in the platform and is frozen for the life of the creator so that Meta ad-name matching stays stable.

Creator SIF ID (single global linear counter)

NEW + HISTORIC

`SIF-{n}` (`inf_id = 'SIF-' || sif_number`)

When: At reach-out, by `submit_reachout` RPC, only the first time a username is seen. Reused thereafter. Frozen forever (Meta ad-name matching depends on it).

Each creator gets one unique id `SIF-{n}`, where `n` is the next integer after the highest `sif_number` already in the creators table. This is the single global creator counter; nothing else in the system is globally numbered this way except campaigns.

LOGIC / SQL

```
submit_reachout (2026_06_25c lines 44-50): perform
pg_advisory_xact_lock(hashtext('reachout-user:' || username)); look up creators by
username; if absent → select coalesce(max(creators.sif_number),0) into v_max; v_new
:= v_max+1; inf_id := 'SIF-' || v_new; insert creator. If present → reuse inf_id and
update profile fields only.
```

EXAMPLES

- `SIF-1` (first creator ever)
- `SIF-2` (second creator)
- `SIF-7829` (registry size as of 2026-06-24)

EDGE CASES & RULES

- Existing creator found by username → SIF reused, never re-minted.
- Historic creators imported with their ORIGINAL legacy SIF frozen (e.g. `SIF-1234`).
- Handle change → creator re-identified by `profile_id` (legacy IG id); SIF unchanged, no renumber.
- `create_repeat_collab` uses the existing creator's SIF; it does not mint a new one.

Creator origin tag

NEW + HISTORIC

Enum: 'historic_creator' | 'new_creator'

When: At creator creation — 'new_creator' via submit_reachout post-launch; 'historic_creator' set during legacy migration.

A flag on the creators row recording whether the creator predates the new platform (migrated) or was first reached out to inside it. Lets queries filter by lineage.

LOGIC / SQL

```
types.gen.ts:138 creators.creator_type enum; reach-out/actions.ts:164 compares creator_type. New reach-outs default to new_creator.
```

EXAMPLES

- historic_creator
- new_creator

EDGE CASES & RULES

- Default for any reach-out submitted after launch is new_creator.
- Historic creators were bulk-tagged during the cleaned_data → creators migration.

2 Post ID (P) — Deliverables

A `post_id` identifies a single deliverable (one reel or one static post). It is minted at onboarding (not reach-out), is per-creator linear, and deliberately carries NO collab suffix — collab grouping is a separate column. This is the segment most often miscopied; the -C suffix belongs to `collab_id`, never to `post_id`.

Deliverable Post ID

NEW + HISTORIC

`{sif_id}-P{post_number}` e.g. SIF-1-P1

When: At onboarding, first onboard only, via `mint_onboarding_block` → `post_id_base`, stamped in `submitOnboarding` (`actions.ts:466`). Re-onboard preserves the stored value. Reach-out rows carry NULL `post_id`.

Each deliverable gets a short id combining the creator's SIF and a per-creator post counter. There is NO -C suffix; collab grouping lives entirely in the separate `collab_id` column. `post_id` and `post_id_short` hold the same value today.

LOGIC / SQL

```
mint_onboarding_block (2026_06_25c:147-157): v_maxp := max(post_number) over (posts
u historic_posts WHERE inf_id=p_inf_id); post_id_base := p_inf_id||'-P'||
(v_maxp+1). submitOnboarding stamps post_id=post_id_short=postIdBase only when
isFirstOnboard (parent.post_id=null).
```

EXAMPLES

- SIF-1-P1 (creator SIF-1, first deliverable across all their collabs)
- SIF-1-P2 (could belong to collab C1 or C2)
- SIF-2-P1 (different creator's first deliverable)

EDGE CASES & RULES

- Reach-out rows: `post_id` NULL, identified by bigserial id.
- Multiple deliverables in one collab take consecutive P numbers ($P\{n\}, P\{n+1\}, \dots$) but ONE shared `collab_id`.
- Per-creator, not global or per-campaign — every creator has their own P1.
- Re-onboarding never re-mints; stored `post_id` preserved.
- Stories increment the deliverable count but never get their own `post_id` row.
- `post_id_short` is a redundant duplicate column (legacy from the pre-2026-06-06 -C-suffix model).

Per-creator linear post number (the 'P')

NEW + HISTORIC

{n} (integer)

When: At onboarding via mint_onboarding_block: start_post_number = max(post_number over posts u historic_posts for this creator)+1. Children take start+1, start+2, ... Reach-out: NULL.

A counter that climbs per creator across all their collabs. Creator SIF-1 gets post_numbers 1,2,3,... no matter which collab each deliverable belongs to. It is the integer inside the P part of post_id.

LOGIC / SQL

```
mint_onboarding_block:147-156 (per-inf_id union max). submitOnboarding:468 assigns
startPostNumber to the parent; child loop (522-591) does
nextPostNum=startPostNumber+1 then nextPostNum++ per child. NO re-query of max
during the child loop (would race the reservation).
```

EXAMPLES

- SIF-1-C1 (1 reel) → post_number 1
- SIF-1-C2 (3 posts) → post_numbers 2,3,4
- SIF-1-C3 (2 reels) → post_numbers 5,6
- Historic continuation: if legacy SIF-1 ended at 50, new C2 starts at P51

EDGE CASES & RULES

- Per-creator, NOT global (current as of 2026-06-25). The pre-2026-06-06 code was global — that is the ONLY era harvest #4's 'GLOBAL' claim was true.
- Reach-out rows NULL until onboarding.
- Children skip the parent's number and take parent+1,+2,...
- Historic posts included in the MAX so live numbering continues from legacy.
- Stories do not increment post_number.

Reach-out row identity (bigserial id)

NEW ONLY

`bigint (1,2,3, ...)`

When: On INSERT into posts at reach-out; auto-incremented. Reach-out rows have NULL `post_id`, so `id` is their only key until onboarding.

Because reach-out rows have no `post_id` yet, they are addressed by the internal bigserial `posts.id`. Once onboarded, the `post_id` is stamped and becomes the external key; `id` stays the true PK.

LOGIC / SQL

```
submit_reachout RETURNING posts.id into v_id; returns it. editReachOut (reach-out/actions.ts:339) keys on rowId when post_id is NULL. stampTestRows keys on id when post_id NULL.
```

EXAMPLES

- First reach-out → `id=1`, `post_id=NULL`, `collab_id=NULL`
- After onboard → `id=1` now has `post_id='SIF-1-P1'`, `collab_id='SIF-1-C1'`

EDGE CASES & RULES

- `UNIQUE(post_id)` permits MULTIPLE NULLs (NULLs distinct in Postgres), so many un-onboarded rows coexist.
- `id` is never NULL; it is the real PK.
- UI shows 'Pending' for NULL-`post_id` rows and never exposes the raw `id`.

Collab ID (C) — One Order = One Collab

A collab represents a single order/agreement. Every deliverable of that order shares one collab_id. Minting is at onboarding and is idempotent on normalized order_id, so re-submitting the same order reuses the same collab. The -C suffix lives ONLY here.

Collaboration grouping id

NEW + HISTORIC

`{sif_id}-C{collab_number}` e.g. SIF-1-C1

When: At onboarding via `mint_onboarding_block`. Reused if the normalized order_id already maps to a collab for this creator; else next per-creator collab_number. Reach-out rows: NULL.

One collab = one order. All deliverables of an order (1 reel, or 3 reels + 2 posts, etc.) share the collab_id. Re-onboarding the same order reuses the same collab_id (idempotent).

LOGIC / SQL

```
mint_onboarding_block (2026_06_25c:121-145): v_oid :=
lower(btrim(regexp_replace(order_id, '^#+', ''))); if v_oid <> '' → reuse existing
collab_number for matching normalized order_id; else v_cn := max(collab_number over
posts u historic_posts for inf_id)+1; collab_id := p_inf_id||'-C'||v_cn. Stamped
submitOnboarding:459.
```

EXAMPLES

- SIF-1-C1 (order #123)
- SIF-1-C2 (order #456)
- SIF-1-C1 reused (order #123 re-onboarded)
- SIF-2-C1 (different creator)

EDGE CASES & RULES

- Order normalization strips leading '#' and trims/lowercases. NOTE: leading zeros are NOT stripped by the regex — '0123' vs '123' equivalence is claimed in harvests but not guaranteed by code; verify.
- Reach-out rows NULL until onboarding.
- Different order_id for same creator → new collab_number minted.
- Re-onboard reuses stored collab_id (submitOnboarding:397-401 short-circuits, does not re-call the RPC).
- One draft payment is raised per collab_id; board KPIs count unique collab_ids.

Per-creator collab episode counter (the 'C')

NEW + HISTORIC

`{n}` (integer ≥ 1)

When: At onboarding via `mint_onboarding_block`. Reused on matching normalized `order_id`; else `max+1` over posts $\cup$ `historic_posts` for the creator. Reach-out: NULL.

A per-creator counter for which collaboration episode this is. SIF-1's first order is C1, second order C2, etc. It is the integer inside the C part of `collab_id`.

LOGIC / SQL

Same derivation as `collab_id` (`mint_onboarding_block:128-145`). Assigned `submitOnboarding:458` (`collab_number=collabNumber`). CHECK `posts_collab_number_chk: collab_number  $\geq 1$` .

EXAMPLES

- SIF-1-C1 (first order)
- SIF-1-C2 (second order)
- Historic continuation: legacy C1-C5 $\rightarrow$ next is C6
- Two deliverables same order $\rightarrow$ same `collab_number`

EDGE CASES & RULES

- Reach-out rows NULL until onboarding.
- Same order re-onboarded $\rightarrow$ stored `collab_number` reused (no increment).
- Historic posts included in MAX so live collabs continue legacy numbering.
- `deliverable_index` is legacy ordering only; `collab_id` is the grouping key.

Position within a collab (legacy ordinal)

NEW + HISTORIC

`{n}` (integer, parent=1, children 2,3,...)

When: At onboarding when total deliverables > 1. Parent=1, children=2,3,... For single-deliverable collabs, parent gets index 1 (see gap).

An ordinal for sorting multiple deliverables inside one collab. Now superseded by `collab_id` for grouping; kept only for ordering within a collab and for historic-table compatibility.

LOGIC / SQL

NEW: `submitOnboarding:477` parent `deliverable_index=1` (firstType branch); child `loop:584` `deliverable_index=childIdx` (starts at 2). HISTORIC: `ROW_NUMBER() OVER (PARTITION BY collab_id ORDER BY post_number)` when `order_id` present, else `NULL`.

EXAMPLES

- Parent reel in C1 → 1; first child post → 2; second child reel → 3
- Single-deliverable collab → 1
- Historic: SIF-153-P1,P2 same order → 1,2

EDGE CASES & RULES

- Set only when total>1 in some paths; value for total=1 is ambiguous (see gaps) — code suggests 1.
- Stories count toward total but never create an indexed child row.
- `deliverable_role` (parent/child/single) is the modern role marker; `deliverable_index` is compatibility-only.
- Historic: `NULL` for all reach-out rows (no `order_id`); all observed multi-post collabs are [1,2] only (no 3+ yet).

Deliverable content format

NEW ONLY

Enum: 'reel' | 'post'

When: At onboarding during deliverable expansion (§6.2). Parent gets firstType; children get the type for each remaining deliverable.

Marks whether a deliverable row is a video reel or a static post. When a collab is onboarded with several deliverables, the parent takes the first type and children spawn with their own types.

LOGIC / SQL

onboarding/actions.ts:404-407 firstType; 475-477 applied to parent; children carry the same/appropriate type.

EXAMPLES

- reel
- post

EDGE CASES & RULES

- Stories are NOT a deliverable_type — story counts are tracked separately and never spawn a typed row (parent.stories is set to 0 at onboarding, line 441).
- A 'single' or 'parent' row has exactly one deliverable_type.

4 Nomenclature (Audit String)

Nomenclature is the human-readable audit label stamped at onboarding. Format is exactly four dash-separated segments. The post_id segment inside it is SIF-N-P{n} with NO -C suffix — the most common harvest error was inserting a -C here.

Nomenclature format string

NEW + HISTORIC

```
{post_id}-{username}-{content_type}-{reach_out_date}
```

When: At onboarding (buildLegacyNomenclature), once post_id is known. Parent and every child get it. Reach-out rows: NULL. Re-onboard preserves an existing value.

A readable identifier joining four parts with dashes: the deliverable post_id (SIF-N-P{n}, NO -C), the creator's Instagram handle, the content-type code, and the date the creator was first reached out to (YYYY-MM-DD, IST).

LOGIC / SQL

```
onboarding/actions.ts:140-149
buildLegacyNomenclature(postId,username,contentType,date) →
`${postId}-${username}-${contentType}-${date}`; returns null if
postId/username/contentType missing. Applied 416-423 (parent, reuses
parent.nomenclature if set) and 542-547 (children). Date = parent.reach_out_date
else todayIsoInIndia().
```

EXAMPLES

- SIF-15-P42-johndoe-Reel-2026-06-15
(CORRECT — no -C)
- SIF-8-P128-fashionista99-Post-2026-05-20
- SIF-1-P1-devesh-UGC-2026-06-15

EDGE CASES & RULES

- CORRECTION: harvest examples like 'SIF-15-P42-C1-johndoe-...' are WRONG — the post_id segment has no -C; the -C is only on the separate collab_id column.
- Reach-out rows NULL nomenclature; identified by bigserial id.
- If username or content_type missing, onboarding builder returns null (row saved without nomenclature).
- editReachOut's builder (reach-out/actions.ts:56-62) does NOT null-check; missing segments would yield a malformed double-dash string.
- contentName is NOT a segment despite the legacy comment at reach-out/actions.ts:306; if enabled the format would gain a 5th segment.
- Date is the ORIGINAL reach_out_date, reused across C2,C3,... not the onboarding date.
- Display falls back to content_type when nomenclature is null (columns.tsx:445-451).

Date segment format

NEW + HISTORIC

YYYY-MM-DD (ISO 8601, Asia/Kolkata)

When: new Date().toLocaleDateString('en-CA',{timeZone:'Asia/Kolkata'}) at reach-out; reused at onboarding.

The date part is an ISO calendar date generated in Indian Standard Time, capturing when outreach happened (not the onboarding moment).

LOGIC / SQL

```
reach-out/actions.ts:50-54 todayIsoInIndia(); onboarding:334-340 nomenclatureDate =  
parent.reach_out_date || today. 'en-CA' locale chosen specifically for YYYY-MM-DD  
output.
```

EXAMPLES

- 2026-06-15
- 2026-05-20

EDGE CASES & RULES

- No time component.
- A reach-out at 11pm UTC may render the next IST day (UTC+5:30).
- Falls back to today if reach_out_date missing.

Username segment

NEW + HISTORIC

lowercase handle (dots/underscores/hyphens preserved, no leading @)

When: At reach-out, extracted from the Instagram link/input and lowercased; frozen once the collab leaves 'Reach Out'.

The creator's Instagram handle without the @, lowercased for case-insensitive uniqueness. Used verbatim as the second nomenclature segment.

LOGIC / SQL

reach-out/actions.ts:44-48 extractUsernameFromInput matches instagram.com/@?([A-Za-z0-9._]+) then lowercases. Stored in creators.username and posts.username; submit_reachout looks up creators by it.

EXAMPLES

- johndoe
- fashionista99
- travelgram

EDGE CASES & RULES

- Frozen after 'Reach Out' status (decision D7); onboarding edits cannot change it.
- Dots/underscores/hyphens preserved.
- Extraction failure → reach-out rejected ('Could not derive username from URL').

Content-type segment

NEW + HISTORIC

Free-text string (no enum)

When: At reach-out; editable via editReachOut while still 'Reach Out' (triggers nomenclature rebuild).
Captured into nomenclature at onboarding.

The deliverable format label (UGC, Reels, Stories, VRP, IFADs, ...). No fixed list — the allowed set is whatever distinct values exist in posts, fetched on demand.

LOGIC / SQL

```
reach-out/schema.ts:63 contentType required string; onboarding/queries.ts:205-213  
fetchOnboardingFilterOptions pulls DISTINCT content_type. editReachOut rebuilds  
nomenclature on content_type change (line 440).
```

EXAMPLES

- UGC
- Reels
- Stories
- VRP
- IFADs
- Plus Size

EDGE CASES & RULES

- No validation against a fixed list; case-sensitive as stored.
- Stories dropped during deliverable expansion; nomenclature uses the parent's type.
- Editable only while in 'Reach Out' (D7 freeze afterward).

5 Campaign ID

Campaigns have their own global identifier namespace, independent of creators. Like SIF, it is a global monotonic counter, advisory-lock serialized.

Campaign ID

NEW ONLY

IFC{NNN} (zero-padded 3 digits)

When: At campaign creation via submit_campaign RPC, advisory-lock serialized.

Campaign ids are IFC + a zero-padded number from MAX(campaign_num)+1 — globally unique and increasing, never reset by year or month.

LOGIC / SQL

```
2026_05_20_submit_campaign_rpc.sql:88 v_campaign_id :=  
'IFC' || lpad(v_campaign_num::text,3,'0'); advisory lock serializes concurrent  
submits.
```

EXAMPLES

- IFC001
- IFC042
- IFC999

EDGE CASES & RULES

- Global counter, no per-period reset.
- Advisory lock prevents concurrent-submit collisions.
- First-value ambiguity (IFC000 vs IFC001) — see gaps; project memory finalized IFC001.

New vs Historic Provenance

New and historic ids share grammar but differ entirely in how they are produced. This section records the historic-table derivation rules and the deliberate numbering continuity between the two.

Historic post_id derivation

HISTORIC

SIF-{sif_number}-P{post_number}

When: Once, at the cleaned_data → historic_posts migration. post_number = ROW_NUMBER() OVER (PARTITION BY username ORDER BY cleaned_data.id).

Each legacy deliverable got a post_id from a per-creator dense row number assigned at migration time, regardless of whether it reached Instagram or had an order. legacy_post_id retains the original sheet value.

LOGIC / SQL

```
SELECT CONCAT('SIF-',cd.sif_id,'-P',ROW_NUMBER() OVER (PARTITION BY cd.username  
ORDER BY cd.id)) FROM cleaned_data cd.
```

EXAMPLES

- SIF-1007-P1, SIF-1007-P2 (two orders)
- SIF-102-P1,P2,P3 (three orders)

EDGE CASES & RULES

- No-order rows still get P{n}.
- Reach-out rows get post_number but NULL collab_id/collab_number/deliverable_index.
- Per-creator (PARTITION BY username), matching the live per-creator rule.

Historic collab_id / collab_number derivation

HISTORIC

SIF-{sif_number}-C{collab_number} / integer

When: At migration, only when order_id present. collab_number = DENSE_RANK() over distinct order_id per creator; NULL for reach-out rows.

Legacy collabs group posts sharing an order_id; each distinct order got a sequential C number by first appearance. Reach-out rows (no order) have NULL collab.

LOGIC / SQL

```
DENSE_RANK() OVER (PARTITION BY sif_id ORDER BY MIN(order_id) OVER (PARTITION BY sif_id,order_id)) when order_id NOT NULL; collab_id = CONCAT('SIF-',sif_id,'-C',collab_number).
```

EXAMPLES

- SIF-102-C1/C2/C3 (three orders)
- SIF-153-P1,P2 same order → both SIF-153-C1

EDGE CASES & RULES

- Same order_id → same collab_id.
- Reach-out rows NULL regardless of post_number.
- 164 Posted-but-no-order backfill rows still NULL collab pending reconciliation.

Historic deliverable_index

HISTORIC

integer from 1

When: At migration when order_id present: ROW_NUMBER() within collab ordered by post_number; NULL otherwise.

Position of a post inside its order group: first=1, second=2. Null for reach-out rows.

LOGIC / SQL

```
ROW_NUMBER() OVER (PARTITION BY collab_id ORDER BY post_number) when order_id NOT NULL.
```

EXAMPLES

- SIF-153-P1→1, SIF-153-P2→2 (same order)
- SIF-102-P1→1 (single-post collab)

EDGE CASES & RULES

- Most collabs are single-post (index 1 only).
- All observed multi-post collabs are [1,2]; none with 3+ deliverables yet.
- NULL for reach-out rows.

Historic workflow_status derivation

HISTORIC

'Posted' | 'On Board' | 'Reach Out'

When: At migration, derived from post_link / order_id presence.

Posted = has a real instagram.com link; On Board = has an order_id but no real link; Reach Out = neither.

LOGIC / SQL

```
CASE WHEN post_link LIKE 'https://www.instagram.com%' THEN 'Posted' WHEN order_id  
IS NOT NULL THEN 'On Board' ELSE 'Reach Out' END. Verified counts: 1,212 Posted  
(1,211 real links), 246 On Board, 9,794 Reach Out.
```

EXAMPLES

- real IG link + order → Posted
- order, link='Ghosted' → On Board
- no order, no link → Reach Out

EDGE CASES & RULES

- 164 Posted-but-no-order anomaly rows pending backfill.
- 19 Reach Out rows carry non-IG post_link values (e.g. 'Ghosted').
- 1 Posted row without a real IG link.

Historic traceback & enrichment columns

HISTORIC

```
source_cleaned_id (bigint FK), profile_id (IG numeric id), legacy_post_id (text)
```

When: At migration; copied 1:1 from cleaned_data.

Archive-only columns absent from live rows: source_cleaned_id links back to the cleaned_data source row; profile_id is the Instagram numeric id (~95% backfilled); legacy_post_id keeps the original sheet post id.

LOGIC / SQL

```
historic_posts.source_cleaned_id = cleaned_data.id (1:1); historic_posts.profile_id  
= cleaned_data.profile_id (NULL where unenriched).
```

EXAMPLES

- SIF-102-P1 → source_cleaned_id=11276
- SIF-1007-P1 → profile_id=8489263759
- SIF-102-P1 → profile_id=NULL (unenriched)

EDGE CASES & RULES

- Every historic row has exactly one source_cleaned_id (no NULL).
- profile_id NULL where legacy enrichment failed (~5%).
- SIF_ERROR rows kept in cleaned_data for audit but EXCLUDED from historic_posts.

New vs Historic, side by side

Same grammar, different provenance — welded together by numbering continuity.

Aspect	New (live posts)	Historic (archive)
Identifier grammar	SIF-{n}-P{m} · SIF-{n}-C{k}	Identical — same grammar
How minted	Transactionally at runtime, under advisory locks	Once, by SQL window functions at the 2026-06-24 migration
SIF id	Minted at reach-out (submit_reachout)	Frozen from the original legacy sheet
P & C minted	At onboarding (mint_onboarding_block)	ROW_NUMBER() / DENSE_RANK() over cleaned_data
post_number	Per-creator; continues over posts u historic	ROW_NUMBER() PARTITION BY username
collab_number	Per-creator; = one order; reuse-on-match else max+1	DENSE_RANK() over distinct order_id
Reach-out rows	NULL post_id — keyed by bigserial id	Carry a migration-assigned post_number; NULL collab
workflow_status	Full 12-value lifecycle enum	Collapsed to Posted / On Board / Reach Out
Archive-only cols	—	source_cleaned_id , profile_id , legacy_post_id
creator_type	new_creator	historic_creator
Continuity weld	MAX(P), MAX(C) UNION over posts u historic — a returning legacy creator continues their numbering (SIF-1 ended P50/C5 → next is P51/C6)	Feeds the live MAX() so numbering never restarts or collides

7 Codes & Enumerations

Closed-set and free-text fields used across the platform, with their exact legal values and the discrepancies engineering should reconcile.

Workflow status (new model)

NEW ONLY

Enum: 'Reach Out' | 'On Board' | 'Order Sent' | 'Posted' | 'Delivered' | 'RTO' | 'C

When: Set at reach-out ('Reach Out'), onboarding ('On Board'), and subsequent transitions.

Tracks a collab's full lifecycle from initial contact through delivery, with terminal failure (RTO/Cancelled) and end-of-life (Offboarding/Offboarded) branches.

LOGIC / SQL

types.gen.ts:21-33 union; status-pill.tsx:44-57 tone mapping. 'On Board' is displayed as 'Onboard' via the label formatter.

EXAMPLES

- Reach Out
- On Board
- Posted
- Delivered
- RTO

EDGE CASES & RULES

- 'On Board' relabeled 'Onboard' in UI.
- RTO / Cancelled / Cancelled After RTO are terminal failure states.
- Awaiting Reply and Declined are rare, not fully integrated into the main flow.

Collaboration type (payment model)

NEW + HISTORIC

Enum: 'Barter' | 'Barter + Paid'

When: At reach-out (outbound) or onboarding (inbound forced to Barter).

Barter = product only (no cash); Barter + Paid = product plus cash. When Barter, commercial_amount is always 0.

LOGIC / SQL

onboarding/schema.ts:27-28 COLLAB_TYPES; Barter rule enforced 2026_06_06...:129-133 and via applyBarterLock.

EXAMPLES

- Barter
- Barter + Paid

EDGE CASES & RULES

- Inbound reach-outs forced to Barter, commercials locked to 0.
- DISCREPANCY: campaign form offers 'Barter'/'Paid'; onboarding historically allowed a third; production rule permits ONLY Barter / Barter + Paid (see gaps).

Reach-out direction vs type

NEW ONLY

```
direction: 'inbound'|'outbound' (lowercase); type: 'Inbound'|'Outbound' (title-case)
```

When: At reach-out; type derived from direction.

reachout_direction (lowercase) is the canonical who-initiated value and drives logic; reachout_type is its title-cased display/audit twin.

LOGIC / SQL

```
schema.ts:41-46 REACHOUT_DIRECTIONS; RPC validates inbound/outbound; actions.ts:166  
p_reachout_type = direction $\equiv$ 'inbound'?'Inbound':'Outbound'.
```

EXAMPLES

- inbound / Inbound
- outbound / Outbound

EDGE CASES & RULES

- Storage of direction is always lowercase.
- Inbound auto-sets collab_type Barter and commercials 0.
- Use reachout_direction for canonical logic, not reachout_type.

Influencer tier by followers

NEW + HISTORIC

Labels: Nano (1K-10K) | Micro (10K-50K) | Mid tier (50K-500K) | Macro (500K-1M) | M

When: GENERATED creators.category, recomputed from followers.

Follower-count buckets for budgeting. WARNING: the form's range labels and the runtime thresholds disagree.

LOGIC / SQL

formatters.ts:82-91 tierFromFollowers uses <10K Nano, <50K Micro, <300K Mid tier, <1M Macro, ≥1M Mega; campaigns/schema.ts:25-31 form labels say 50K-500K for Mid tier.

EXAMPLES

- Nano (1K to 10K)
- Mid tier (50K to 500K)
- Mega (1M+)

EDGE CASES & RULES

- DISCREPANCY: form label 'Mid tier 50K-500K' vs code boundary 300K. A 400K creator is labeled Mid tier by the form but Macro by the GENERATED column. Reconcile (see gaps).
- category recomputed whenever followers change.

Content type (free-text)

NEW + HISTORIC

Free-text (no enum)

When: At reach-out.

Operator-entered label; no fixed list. The 'allowed' set is whatever distinct values already exist in posts, fetched on demand.

LOGIC / SQL

```
onboarding/queries.ts:205-213 DISTINCT content_type from posts.
```

EXAMPLES

- UGC
- Reels
- Stories
- VRP
- IFADs

EDGE CASES & RULES

- No validation; case-sensitive.
- Distinct set is data-driven, not hard-coded.

8 Concurrency, Integrity & Invariants

The rules that keep id generation race-free and the data internally consistent across both reach-out and onboarding paths.

Advisory-lock concurrency model

NEW + HISTORIC

N/A (mechanism)

When: At reach-out (`submit_reachout`) and onboarding (`mint_onboarding_block`). Locks held to transaction commit.

PostgreSQL advisory locks serialize id generation. Reach-out locks on the username (prevents duplicate creators). Onboarding locks on the creator (serializes the P-block + C reservation). Locks release at transaction end.

LOGIC / SQL

```
submit_reachout (2026_06_25c:43): pg_advisory_xact_lock(hashtext('reachout-user:'||username)) before creator upsert. mint_onboarding_block:126:
pg_advisory_xact_lock(hashtext('reachout-inf:'||p_inf_id)) before computing
maxP/maxC. hashtext() → stable bigint.
```

EXAMPLES

- Two reach-outs for 'alice': first wins `reachout-user:alice` and creates SIF-123; second waits, sees SIF-123, skips creation.
- Two onboards for SIF-123 (different orders): first locks `reachout-inf:SIF-123`, reserves P6&C2; second waits, then sees P6 written and reserves P7&C3.

EDGE CASES & RULES

- CORRECTION to harvest #1: the current reach-out path takes ONLY the 'reachout-user' lock; the second 'reachout-inf' lock at reach-out time was removed — it now lives in `mint_onboarding_block` (onboarding-time only).
- Locks are transaction-scoped; rollback releases before any dependent txn sees new ids.
- `hashtext` collisions theoretically possible but extremely rare.
- Locks serialize writes only; reads are not blocked.
- No deadlock: both RPCs acquire in a consistent order.

Barter = 0 commercial amount

NEW + HISTORIC

N/A (invariant)

When: At reach-out and onboarding whenever collab_type='Barter'.

Barter collabs always have commercial_amount 0, enforced at both the database (RPC) and app (Zod/applyBarterLock) layers to keep data consistent.

LOGIC / SQL

```
submit_reachout (2026_06_25c:62-63): if p_collab_type='Barter' then  
  v_commercial_amount:=0; else coalesce(p_commercial_amount,0). submitOnboarding:179  
applyBarterLock(parsed.data) zeros it pre-validation; per-deliverable amount  
derived from the already-locked value.
```

EXAMPLES

- Barter + input 5000 → stored 0
- Barter + Paid → commercial_amount > 0 allowed

EDGE CASES & RULES

- Inbound batch passes collab_type=null at reach-out, so the lock applies only at onboarding.
- Equal-split of 0 across N deliverables = 0 each (consistent).
- Payment rows still created for Barter (amount 0, status 'Not Due', never 'Due').

Equal-split per-deliverable commercials

NEW ONLY

```
perDeliverableAmount = round(agreed_total / total_deliverables, 2)
```

When: At onboarding, stored on every deliverable row of the collab.

The agreed collab total is divided equally across all deliverables and stored per row, so SUM(commercial_amount) over the collab reconstructs the original agreed total.

LOGIC / SQL

```
onboarding/actions.ts:412-413 perDeliverableAmount = total>0 ?  
Math.round((v.commercials/total)*100)/100 : v.commercials; applied to parent (432)  
and children.
```

EXAMPLES

- 10000 over 4 deliverables → 2500 each
- Barter (0) over 3 → 0 each

EDGE CASES & RULES

- Rounding to 2dp may leave tiny SUM drift vs the exact total on non-divisible amounts.
- Barter forces the total to 0 first, so split is trivially 0.

Order-ID normalization for collab matching

NEW + HISTORIC

```
lower(btrim(regexp_replace(order_id, '^#+', '')))
```

When: At onboarding inside mint_onboarding_block before collab lookup.

Before deciding whether an order already has a collab, the order_id is canonicalized: strip leading '#', trim whitespace, lowercase. Matching ids map to the same collab.

LOGIC / SQL

```
mint_onboarding_block:121,133 apply the same regexp/btrim/lower to both the  
incoming order_id and stored order_ids.
```

EXAMPLES

- '#123' → '123'
- ' ABC ' → 'abc'
- '#123' and '123' → same collab

EDGE CASES & RULES

- GAP: leading-zero stripping ('0123'→'123') is asserted in harvest #1 examples but the regex only removes '#', not leading zeros — verify before relying on it.
- Empty/whitespace order_id (v_oid='') skips reuse and always mints a fresh collab_number.

Re-onboard idempotency

NEW ONLY

N/A (invariant)

When: At onboarding, when parent.post_id is already set (isFirstOnboard=false).

Re-onboarding an already-minted row never re-mints ids or nomenclature; the stored post_id, post_number, collab_id, collab_number and nomenclature are preserved.

LOGIC / SQL

```
submitOnboarding:369 isFirstOnboard = parent.post_id==null. When false (397-401) it  
reuses parent.post_number and parent.collab_id and does NOT call  
mint_onboarding_block; the post_id/post_number/post_id_short writes are guarded by  
`if (isFirstOnboard)` (465-469).
```

EXAMPLES

- First onboard of SIF-1 row → mints SIF-1-P1 / SIF-1-C1
- Edit-resubmit same row → keeps SIF-1-P1 / SIF-1-C1, no new mint

EDGE CASES & RULES

- nomenclature reused via `parent.nomenclature || build...` (416-417).
- Children are spawned only on first onboard when total>1 (515).

Glossary

Every term used across the platform's identifier system, defined precisely.

SIF

Saadaa Influencer Framework. The brand's internal creator-identity namespace. Every creator gets one SIF id of the form SIF- $\{n\}$; it is the root prefix of every `post_id` and `collab_id`.

inf_id

The creator's SIF identifier as stored in the database (`creators.inf_id`, `posts.inf_id`). Equals 'SIF-' || `sif_number`. Stable forever once minted.

sif_number

The bare integer behind a SIF id (`creators.sif_number`). The system's single global monotonic creator counter; $\max(\text{sif_number}) + 1$ mints the next creator.

post_id

Deliverable identifier, format SIF- $\{n\}$ -P $\{m\}$. One per reel or static post. Minted at onboarding. Carries NO -C suffix — collab grouping is a separate column.

post_id_short

A second column holding the identical value to `post_id` in the current model. Legacy remnant from the pre-2026-06-06 era when `post_id` carried a -C suffix and `post_id_short` did not. Effectively redundant now.

post_number

The integer 'P' component of `post_id`. Per-creator linear counter; continues across a creator's live posts $\cup$ `historic_posts` (both filtered by `inf_id`). NOT global as of 2026-06-25.

collab_id

Collaboration (one order = one collab) identifier, format SIF- $\{n\}$ -C $\{k\}$. All deliverables sharing one normalized `order_id` share this id. Minted/reused at onboarding.

collab_number

The integer 'C' component of `collab_id`. Per-creator episode counter; reused when the same normalized `order_id` reappears, else $\max + 1$ over posts $\cup$ `historic_posts`. CHECK constraint: ≥ 1 .

collab episode

One discrete agreement/order with a creator, regardless of how many deliverables it contains. Synonymous with one `collab_id` / one `collab_number` value.

deliverable_index

Legacy ordinal for ordering multiple deliverables within one collab (`parent=1`, `children=2,3,...`). Superseded by `collab_id` for grouping; retained for sort order and historic compatibility.

deliverable_role

Modern role tag on each post row: 'parent' | 'child' | 'single'. Replaces the old parent/child model; collab_id is the true grouping key.

deliverable_type

Content format of one deliverable row. Enum strictly 'reel' | 'post'. Stories are counted but never spawn a typed row.

nomenclature

Human-readable audit string: {post_id}-{username}-{content_type}-{reach_out_date}. Built at onboarding via buildLegacyNomenclature. Nullable; reach-out rows have NULL.

mint_onboarding_block

The RPC that, in one advisory-locked call, reserves a collab (reuse-or-new) and a contiguous post_number block for a creator at onboarding. Returns collab_number, collab_id, start_post_number, post_id_base.

submit_reachout

The RPC that creates (or reuses) a creator and inserts a reach-out posts row with NULL post_id/collab. Mints the SIF for brand-new creators. Holds advisory lock on 'reachout-user:{username}'.

create_repeat_collab

RPC that inserts a fresh Reach Out posts row for an EXISTING creator (C2+ scenario), with NULL post_id/collab — the new collab is minted later at onboarding.

order_id normalization

Canonicalization applied before collab matching: lower(btrim(regexp_replace(order_id, '^#+', ''))). Makes '#123', '0123' → '123' equivalence... note: leading-zero stripping is NOT done by the regex (only '#' and whitespace); '0123' vs '123' equivalence is asserted in harvest examples but not guaranteed by the shown regex — verify.

advisory lock

pg_advisory_xact_lock(hashtext(key)) — a transaction-scoped Postgres lock used to serialize per-creator / per-username ID generation and prevent duplicate-id races. Released at commit/rollback.

Barter rule

Hard invariant: collab_type='Barter' forces commercial_amount=0, enforced at both RPC (submit_reachout) and app (applyBarterLock) layers.

collab_type

Payment model of a collab: 'Barter' (product only) or 'Barter + Paid' (product + cash). Inbound reach-outs are always Barter.

reachout_direction

Lowercase enum 'inbound' | 'outbound' — who initiated contact. Inbound forces Barter + 0 commercials.

reachout_type

Title-cased display label 'Inbound' | 'Outbound', derived from reachout_direction for human-readable reports.

creator_type

Origin tag on creators: 'historic_creator' (migrated from legacy) | 'new_creator' (created post-launch via submit_reachout).

campaign_id

Campaign identifier, format IFC{NNN} (zero-padded to 3 digits) from MAX(campaign_num)+1. Global, monotonic, advisory-lock serialized.

influencer_tier / category

GENERATED creators.category bucketing by follower count (Nano/Micro/Mid tier/Macro/Mega). Runtime thresholds (300K Mid-tier boundary) differ from campaign-form labels (500K) — known discrepancy.

content_type

Free-text deliverable label (UGC, Reels, Stories, VRP, IFADs, ...). No fixed enum; distinct set derived dynamically from posts at filter-fetch time. Appears as a nomenclature segment.

historic_posts

Archive table (~11,252 rows from ~7,955 creators) of legacy deliverables, ids derived deterministically from cleaned_data via window functions at the 2026-06-24 migration.

cleaned_data

Staging table holding the canonicalized legacy Google-Sheet rows; source of every historic_posts field via source_cleaned_id FK. Holds SIF_ERROR audit markers.

source_cleaned_id

FK on historic_posts pointing back to cleaned_data.id (1:1). Enables full traceback of any archived id to its derivation inputs.

profile_id

Instagram numeric account id (legacy ig_id), copied from cleaned_data; ~95% backfilled, NULL where enrichment failed. Used to re-identify creators across handle changes.

SIF_ERROR

A sentinel in cleaned_data.sif_id marking a row that failed canonical-SIF dedup/assignment. Kept in cleaned_data for audit; EXCLUDED from historic_posts (not promoted to a named post).

workflow_status

Lifecycle stage of a posts row. New model: 12-value enum (Reach Out → On Board → Order Sent → Posted → Delivered, plus RTO/Cancelled/Offboarding/etc.). Historic model: only Posted | On Board | Reach Out.

bigserial id

posts.id — the true primary key, auto-incremented per row. Sole identifier for reach-out rows while post_id is NULL.



Known discrepancies & transient state

An honest source of truth records what is *not* yet clean. These are flagged for engineering to reconcile — do not treat them as settled rules.

⚠ Influencer tier: 300K vs 500K boundary

The campaign form labels `Mid tier` as 50K–500K, but the runtime `tierFromFollowers` (`formatters.ts`) uses a **<300K** cutoff. A 400K-follower creator is labelled **Mid tier** by the form yet classified **Macro** by the generated `creators.category` column. Engineering must pick one boundary.

⚠ collab_type: is bare 'Paid' legal?

The production rule + project memory allow only **Barter** and **Barter + Paid**. The campaign form still surfaces a `Paid` option (most likely a legacy mislabel of Barter + Paid). Reconcile the campaign-schema enum with the onboarding enum.

⚠ post_id_short duplicates post_id

Both columns hold the identical value today. `post_id_short` is a remnant of the pre-2026-06-06 model where `post_id` carried a `-C` suffix and the short form did not. Treat it as deprecated/redundant — never rely on a difference between the two.

⚠ order_id normalization does NOT strip leading zeros

Collab matching canonicalizes the `order_id` as `lower(trim(regex_replace(order_id, '^#+', '')))` — it strips a leading `#` and whitespace, lowercases, but does **not** strip leading zeros. `0123` and `123` are **not** guaranteed to map to the same collab. Verify before relying on it.

⚠ Transient: 164 Posted-but-no-order historic rows

164 historic `Posted` rows have a real Instagram link but `NULL order_id` → `NULL collab`. The team is backfilling `order_ids` (export → fill → re-ingest → Shopify sync). Once done these rows gain proper `collab_ids` and the anomaly disappears. As of 2026-06-25 treat their collab fields as pending.

⚠ SIF_ERROR rows live only in cleaned_data

2,178 dedup-failure rows (SIF_ERROR sentinel) are kept in cleaned_data for audit but are **excluded** from historic_posts (never promoted to a named post). The clean archive is 11,252 of the 13,430 cleaned_data rows.

⚠ Latent 5th nomenclature segment (contentName)

The reach-out form has a contentName field flagged 'legacy nomenclature segment' but it is NOT in the current 4-part nomenclature. If ever enabled, the format would become {post_id}-{username}-{content_type}-{content_name}-{date} .

⚠ Campaign IDs start at IFC001

Format IFC{NNN} from MAX(campaign_num)+1 , zero-padded to 3 digits. Project memory finalized the first campaign as **IFC001** (1-indexed), not IFC000.

Saadaa CreatorHub — Master ID & Nomenclature Guide. Compiled from the live RPCs (submit_reachout, mint_onboarding_block, create_repeat_collab), the onboarding/reach-out server actions, the historic_posts migration, and the live Supabase schema.

Canonical rule: **collab = one order; P and C both minted at onboarding; P/C continue over posts u historic_posts.** Keep this file in sync with project_collab_deliverable_numbering_rule and KB ch04/ch06.